

# Francisco Rubio Illán

frubio-i

[frubio-i@student.42barcelona.com](mailto:frubio-i@student.42barcelona.com)

## SO\_LONG

### Subject

**Archivos a entregar:** Makefile, \*.h, \*.c, maps, textures.

**Makefile:** all, clean, fclean, re, ... Sin relink en ningún caso.

**Argumentos:** un mapa en formato \*.ber.

#### Funciones autorizadas:

- open, close, read, write, malloc, free, perror, strerror, exit.
- Todas las funciones de la librería math (flag -lm).
- Todas las funciones de la miniLibX/MLX42.
- ft\_printf y cualquier función equivalente programada por nosotros.

Se permite usar **libft**: Yes.

**Descripción:** crea un juego 2D usando miniLibX/MLX42. Proyecto gráfico.

#### Objetivos:

- **Jugador** debe recolectar todos los **objetos** del mapa para poder salir.
- **Movimiento** de personaje con **WASD**, **ZQSD** o teclas de **dirección**.
- **No** puedes **traspasar** las **paredes**, debe estar delimitado.
- **Número** de **movimientos** reales deben ser **mostrados** en terminal (en pantalla para bonus).
- Debes usar una **perspectiva 2D** (vista de pájaro o lateral).
- **No** tiene por qué funcionar en **tiempo real**.
- La **temática** es totalmente **libre**.

### Gráficos:

- El programa mostrará la **imagen** en una **ventana**.
- La **gestión** de tu ventana debe ser **limpia** (cambiar de ventana, minimizar, etc).
- Pulsar la tecla **ESC**/pulsar la **cruz** roja debe **cerrar** la **ventana** y cerrar el programa limpiamente.
- El uso de **images** de la miniLibX/MLX42 es **obligatorio**.

### Mapa:

- Consta de 3 **componentes**: paredes, objetos y espacio abierto.
- Debe tener 5 **caracteres** (**0** para espacio vacío, **1** muro/pared, **C** coleccionable, **E** salida de mapa y **P** posición inicial de jugador).
- Debe tener una **salida**, una **posición inicial** y al menos un **objeto**.
- **No** debe haber **más** de una salida (**E**) o posición inicial (**P**), se debe de mostrar un mensaje de error en su caso.
- Debes **comprobar** si hay un **camino válido** en el mapa (Flood Fill).
- Debes poder **procesar cualquier mapa** si sigue todas las reglas correctamente.
- Debes manejar **errores**. Los **fallos** de configuración de archivo deben terminar correctamente y mostrar **"Error\n"** seguido de un **mensaje** de tu elección.

Ejemplo de **mapa**:

```
11111111111111
100100000000C1
1000011111001
1P0011E000001
11111111111111
```

### BONUS

- Conseguir que jugador pierda cuando es atacado o toca a un enemigo/trampa.
- Conseguir añadir algunas animaciones de sprites.
- Conseguir mostrar el contador de movimientos reales directamente en pantalla en lugar de terminal.

# LIBRERIA MLX42 (CODAM)

**MLX42** es una **biblioteca gráfica** de código abierto desarrollada por **Codam Coding College** como una **alternativa** moderna y funcional a la **MiniLibX** (anteriormente usada, basada en X Window). Está diseñada para ser una biblioteca gráfica **simple** y **multiplataforma** que utiliza **GLFW** y **OpenGL**.

## Características principales de MLX42:

- **Multiplataforma:** Compatible con sistemas operativos como Windows, macOS y Linux, permitiendo el desarrollo de aplicaciones gráficas en diversos entornos.
- **Interfaz simplificada:** Proporciona una API sencilla para la creación y gestión de ventanas, manejo de eventos de entrada (teclado y ratón) y renderizado de gráficos 2D.
- **Soporte para texturas y shaders:** Facilita la carga y manipulación de texturas, así como la utilización de shaders para efectos gráficos avanzados.
- **Basada en GLFW y OpenGL:** Aprovecha las capacidades de GLFW para la gestión de ventanas y eventos, y de OpenGL para el renderizado gráfico, garantizando un rendimiento eficiente.

# Estructura de repositorio

└─ [codam-coding-college-MLX42/](#)

- └─ README.md
- └─ CMakeLists.txt
- └─ CODE\_OF\_CONDUCT.md
- └─ CONTRIBUTING.md
- └─ LICENSE
- └─ SECURITY.md
- └─ **cmake/**
  - | └─ Findglfw3.cmake
  - | └─ LinkGLFW.cmake
- └─ **docs/**
  - | └─ 42.md
  - | └─ Basics.md
  - | └─ Colors.md
  - | └─ Functions.md
  - | └─ Hooks.md
  - | └─ Images.md
  - | └─ Input.md
  - | └─ Shaders.md
  - | └─ Textures.md
  - | └─ XPM42.md
  - | └─ index.md
  - | └─ assets/
- └─ **include/**
  - | └─ **KHR/**
    - | └─ khrplatform.h
  - | └─ **MLX42/**
    - | └─ MLX42.h
    - | └─ MLX42\_Int.h
  - | └─ **glad/**
    - | └─ glad.h

```
|   └─ lodepng/
|       └─ lodepng.h
└─ lib/
    └─ glad/
        └─ glad.c
    └─ png/
        └─ lodepng.c
└─ shaders/
    └─ default.vert
└─ src/
    └─ mlx_cursor.c
    └─ mlx_exit.c
    └─ mlx_images.c
    └─ mlx_init.c
    └─ mlx_keys.c
    └─ mlx_loop.c
    └─ mlx_monitor.c
    └─ mlx_mouse.c
    └─ mlx_put_pixel.c
    └─ mlx_window.c
    └─ font/
        └─ font.h
        └─ mlx_font.c
    └─ textures/
        └─ mlx_png.c
        └─ mlx_texture.c
        └─ mlx_xpm42.c
    └─ utils/
        └─ mlx_compare.c
        └─ mlx_error.c
        └─ mlx_list.c
        └─ mlx_utils.c
```

- └─ **tests/**
  - | └─ CMakeLists.txt
  - | └─ WindowFixture.hpp
  - | └─ tests.cpp
- └─ **tools/**
  - | └─ compile\_shader.bat
  - | └─ compile\_shader.sh
  - | └─ xpm3\_conv.py
- └─ **web/**
  - | └─ README.md
  - | └─ coi-serviceworker.js
  - | └─ demo.js
  - | └─ demo.wasm
  - | └─ index.html
- └─ **.github/**
  - └─ **ISSUE\_TEMPLATE/**
    - | └─ bug\_report.md
    - | └─ feature\_request.md
  - └─ **workflows/**
    - └─ ci.yml
    - └─ wasm.yml

# Resumen de las funciones

## Main functions

### mlx\_cursor.c

- **mlx\_set\_cursor\_mode(mlx\_t\* mlx, int mode):** Establece el modo del cursor en la ventana, permitiendo configuraciones como cursor normal, oculto o deshabilitado.
- **mlx\_get\_cursor\_pos(mlx\_t\* mlx, double\* xpos, double\* ypos):** Obtiene la posición actual del cursor en la ventana y la almacena en las variables proporcionadas.
- **mlx\_set\_cursor\_pos(mlx\_t\* mlx, double xpos, double ypos):** Establece la posición del cursor en la ventana en las coordenadas especificadas.

### mlx\_exit.c

- **mlx\_close\_window(mlx\_t\* mlx):** Cierra la ventana actual y libera los recursos asociados.
- **mlx\_terminate(mlx\_t\* mlx):** Finaliza la instancia de MLX42, liberando todos los recursos y memoria utilizados.

### mlx\_images.c

- **mlx\_new\_image(mlx\_t\* mlx, int width, int height):** Crea una nueva imagen con las dimensiones especificadas y la asocia a la instancia de MLX42.
- **mlx\_delete\_image(mlx\_t\* mlx, mlx\_image\_t\* image):** Elimina una imagen previamente creada y libera la memoria asociada.
- **mlx\_image\_to\_window(mlx\_t\* mlx, mlx\_image\_t\* image, int x, int y):** Dibuja una imagen en la ventana en las coordenadas especificadas.

### mlx\_init.c

- **mlx\_init(int width, int height, const char\* title, bool fullscreen):** Inicializa una nueva instancia de MLX42, creando una ventana con las dimensiones y título especificados, y opcionalmente en modo pantalla completa.

## **mlx\_keys.c**

- **mlx\_is\_key\_down(mlx\_t\* mlx, int key)**: Verifica si una tecla específica está siendo presionada en el momento de la llamada.
- **mlx\_key\_hook(mlx\_t\* mlx, void (\*func)(int key, void\* param), void\* param)**: Establece una función de callback que se llama cuando se detecta una pulsación de tecla.

## **mlx\_loop.c**

- **mlx\_loop(mlx\_t\* mlx)**: Inicia el bucle principal de la aplicación, gestionando eventos y renderizando la ventana hasta que se solicite su cierre.
- **mlx\_loop\_hook(mlx\_t\* mlx, void (\*func)(void\*), void\* param)**: Establece una función de callback que se llama en cada iteración del bucle principal, permitiendo actualizaciones periódicas.

## **mlx\_monitor.c**

- **mlx\_get\_monitor\_size(int\* width, int\* height)**: Obtiene las dimensiones del monitor principal y las almacena en las variables proporcionadas.

## **mlx\_mouse.c**

- **mlx\_is\_mouse\_down(mlx\_t\* mlx, int button)**: Verifica si un botón específico del ratón está siendo presionado en el momento de la llamada.
- **mlx\_mouse\_hook(mlx\_t\* mlx, void (\*func)(int button, int action, int mods, void\* param), void\* param)**: Establece una función de callback que se llama cuando se detecta un evento de ratón, como clics o movimientos.

## **mlx\_put\_pixel.c**

- **mlx\_put\_pixel(mlx\_image\_t\* image, int x, int y, uint32\_t color)**: Dibuja un píxel de un color específico en las coordenadas (x, y) de una imagen.



## **mlx\_window.c**

- **mlx\_set\_window\_title(mlx\_t\* mlx, const char\* title):** Cambia el título de la ventana a la cadena de caracteres especificada.
- **mlx\_get\_window\_size(mlx\_t\* mlx, int\* width, int\* height):** Obtiene las dimensiones actuales de la ventana y las almacena en las variables proporcionadas.
- **mlx\_set\_window\_size(mlx\_t\* mlx, int width, int height):** Establece nuevas dimensiones para la ventana.

## **Font/**

### **mlx\_font.c**

- **mlx\_load\_font(mlx\_t\* mlx, const char\* font\_path):** Carga una fuente desde el archivo especificado para su uso en la renderización de texto.
- **mlx\_draw\_text(mlx\_t\* mlx, const char\* text, int x, int y, uint32\_t color):** Dibuja el texto proporcionado en las coordenadas (x, y) de la ventana con el color especificado.

## **Textures/**

### **mlx\_png.c**

- **mlx\_load\_png(mlx\_t\* mlx, const char\* file\_path):** Carga una imagen PNG desde el archivo especificado y la convierte en una textura utilizable en MLX42.

### **mlx\_texture.c**

- **mlx\_create\_texture(mlx\_t\* mlx, int width, int height):** Crea una nueva textura con las dimensiones especificadas.
- **mlx\_delete\_texture(mlx\_t\* mlx, mlx\_texture\_t\* texture):** Elimina una textura previamente creada y libera la memoria asociada.

### **mlx\_xpm42.c**

- **mlx\_load\_xpm42(mlx\_t\* mlx, const char\* file\_path):** Carga una imagen en formato XPM42 desde el archivo especificado y la convierte en una textura utilizable en MLX42.

## Utils/

### mlx\_compare.c

- **mlx\_compare(const void\* a, const void\* b):** Compara dos valores o estructuras según criterios definidos, útil para operaciones de ordenamiento o búsqueda dentro de la biblioteca.

### mlx\_error.c

- **mlx\_error(const char \*message):**  
Muestra un mensaje de error personalizado y, en algunos casos, finaliza la ejecución del programa para evitar un comportamiento inesperado.

### mlx\_list.c

- **mlx\_list\_add():**  
Añade un elemento a una lista específica utilizada internamente por MLX42.
- **mlx\_list\_remove():**  
Elimina un elemento de una lista gestionada por MLX42.
- **mlx\_list\_clear():**  
Limpia y libera todos los elementos de una lista.

### mlx\_utils.c

- **mlx\_clamp(int value, int min, int max):**  
Restringe un valor para que esté dentro de un rango especificado.
- **mlx\_min(int a, int b):**  
Devuelve el menor de dos valores.
- **mlx\_max(int a, int b):**  
Devuelve el mayor de dos valores.
- **mlx\_swap(void\* a, void\* b):**  
Intercambia los valores de dos variables.

# Funciones propias del proyecto:

## [SO\_LONG – Design]

### WANNABE

**so\_long.c** (entrada programa. Creacion de level, por fd o argv[1], depende)

- **open\_fd** (funcion que abre fd de argv[1] o de maps[i] predeterminados)
- **maps\_init** (inicializacion de mapas predeterminados)
- **display\_moves\_colls** (funcion que muestra por pantalla movimientos y colleccionables)

**level.c**

- **ft\_columns\_count** (calcula cols)
- **ft\_level\_count** (calcula rows)
- **ft\_conv\_level** (aloca en memoria mapa con rows y cols usando gnl)
- **player\_coll\_pos** (determina y guarda posicion inicial de player y colleccionables)

**check\_level.c** (parseo del level.map, validacion)

- **check\_wall** (comprueba que todo este rodeado por borders/wall, por 1)
- **check\_player** (comprueba que haya un jugador y no mas, P)
- **check\_exit** (comprueba que haya una salida y no mas, E)
- **check\_char\_valid** (comprueba que todo sea 0, 1, P, E o C)

**check\_path.c**

- **check\_level** (hace todas las comprobaciones de check\_level y tambien si hay un camino valido)
- **dfs** (Depth-first Search. Algoritmo recursivo que comprueba si hay un path valido en el juego)
- **copy\_level** (funcion usada para copiar map en otro doble puntero para no cambiarlo)
- **is\_valid\_path** (funcion que ejecuta todo lo anterior en busca de un camino valido recogiendo colleccionables tambien)

**game.c** (maneja el inicio del juego y queda en espera de inputs)

- **display\_info** (funcion que muestra en terminar resolucion, tile y posiciones de exit y player)
- **game\_init** (funcion donde se ejecuta la mlx y se inicia el juego despues del parseo+checks)

**game\_utils.c**

- **win\_borders** (que se ajuste la pantalla al mapa introducido, por decidir)
- **ft\_strcpy** (strcpy modificado para so\_long)
- **init\_s** (inicia todas las variables de las estructuras que se van a usar)
- **scale\_monitor** (funcion que inicia una mlx y la borra solo para recopilar la resolucion de la pantalla donde se va a ejecutar el juego)
- **tile\_size** (funcion que calcula el size del tile definido por la resolucion de la pantalla. De manera dinamica)

**displayPlayer.c**

- **mirrored\_player** (cambia layers entre normal y mirrored)
- **sprites\_player** (carga sprites de player, normal y mirrored)
- **set\_player** (llamada desde display\_player, muestra el sprite en la posicion inicial del player)
- **display\_player** (recorre el mapa hasta la posicion inicial del jugador y llama a set\_player)

**displayEnv.c**

- **sprite\_background** (carga sprites de background)
- **display\_background** (recorre el mapa hasta poniendo sprite de bg en todo)
- **sprite\_borders** (carga sprites de hole)
- **display\_borders** (recorre el mapa hasta poniendo sprite de borders donde haya 1)

**displayCollectibles.c**

- **sprite\_collectibles** (carga sprites de collectibles)
- **display\_collectibles** (recorre el mapa hasta poniendo sprite de colleccionables donde haya C y guarda su posicion y les asigna un index)

## **displayExit.c**

- **sprite\_hole** (carga sprites de hole)
- **display\_hole** (salida bloqueada sin collectibles recogidos)
- **sprite\_exit** (carga sprites de exit)
- **display\_exit** (bool condicion todos collectibles recogidos)

## **playerController.c**

- **player\_input** (hook desde donde se llama a cada move y detecta con **is\_collider** si esta permitido hacerlo, tambien detecta ESC e imprime movimienos, FPS y colleccionables restantes)
- **move\_up**
- **move\_down**
- **move\_left**
- **move\_right**

## **interactions.c** (basarse en coordenadas de mapa, por unidades, mas sencillo para detectar)

- **is\_collider** (detecta si hay colision y puedes moverte en esa direccion)
- **check\_move** (interaccion con pared, collectibles, exit, enemigo, trampas ...)
- **is\_wall** (detecta si hay pared/border, no usado porque se hace collider y no traspasa)
- **is\_coll** (detecta cada colleccionable, comprueba cual es y lo manda al fondo)
- **is\_exit** (detecta hole antes de recoger colleccionables y exit cuando se puede salir)

## **error\_exit\_free.c**

- **close\_callback** (hook para salir cuando se detecta que se pulsa la X de la ventana, y la cierra limpiamente)
- **exit\_error** (salida con error mediante con mensaje personalizado y liberacion de memoria)
- **exit\_game** (salida del juego con mensaje personalizado y liberacion de memoria y mlx)

## Consideraciones personales

Ha sido un proyecto muy interesante donde he podido por primera vez diseñar antes de programar, lo que me ha permitido tener todo bajo control. También ha sido la primera vez manejando una librería gráfica, lo que ayuda a ver un resultado más atractivo de todo lo que hacemos, o al menos a mi me lo ha parecido.

También, he tenido que controlarme y basar mi esfuerzo en el tiempo que he tenido, con el deadline. No haciendo así el bonus aún teniendo pensado hacerlo. Creo que volveré en algún punto al proyecto o haré un port e iteraré en Godot/Unity. Veremos.

Ahora, a por la siguiente milestone. Allá que voy Minishell.